Data Analytics with Python and Data Storage Reflection

Student Name: Mariam Alobaidli
Student ID: 12702766
Module: Launching into Computing
Assignment: Data Analytics Task
University: University of Essex
Date: 30/03/2026

1. Introduction

This report presents a data analytics task using Python on a weather dataset for Qatar in 2025. The data was obtained from a publicly available source using the Open-Meteo API and includes information such as temperature and rainfall.

The aim of this analysis is to explore weather patterns by loading, cleaning, and analyzing the data using Pandas, and creating charts using Matplotlib. The dataset was organized into monthly values to make trends easier to understand. Basic statistical measures such as mean and median were also calculated.

The report also includes a reflection comparing different data storage models, focusing on SQL and NoSQL, and how they relate to the structure and use of the dataset.

2. Python Implementation

2.1 Data Loading

```python
# Import libraries
import pandas as pd
import requests

# Load data from Open-Meteo API
url = "https://archive-api.open-meteo.com/v1/archive?latitude=25.29&longitude=51.53&start_date=2025-01-
01&end_date=2025-12-
31&daily=temperature_2m_max,temperature_2m_min,precipitation_sum&timezone=Asia%2FQatar"

response = requests.get(url)
weather_json = response.json()

# Convert to DataFrame
df = pd.DataFrame(weather_json['daily'])

df.head()
```

Description: The code uses the requests library to retrieve data from an external API and converts it into a structured format using pandas, allowing further analysis and visualization.

2.2 Data Cleaning

```python
# Rename columns
df.columns = ['date', 'max_temp', 'min_temp', 'rain']

# Convert date column
df['date'] = pd.to_datetime(df['date'])

# Check for missing values
print(df.isnull().sum())

# Check for duplicates
print("Duplicate rows:", df.duplicated().sum())

# Remove duplicate rows
df = df.drop_duplicates()
df.head()
```

Description: This code prepares the dataset by renaming columns, converting the date column into a proper datetime format, and checking for missing values and duplicate rows. Duplicate entries are then removed to ensure the dataset is clean and ready for accurate analysis.

2.3 Aggregation

```python
# Create month column
df['month_name'] = df['date'].dt.strftime('%b')

# Group data by month
monthly_df = df.groupby('month_name').agg({
    'max_temp': 'mean',
    'min_temp': 'mean',
    'rain': 'sum'
}).reset_index()

# Sort months
month_order = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
               'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

monthly_df['month_name'] = pd.Categorical(monthly_df['month_name'], categories=month_order,
ordered=True)
monthly_df = monthly_df.sort_values('month_name')
monthly_df = monthly_df.reset_index(drop=True)

monthly_df = monthly_df.round(2)

monthly_df
```

Description: The data is aggregated monthly by calculating average temperatures and total rainfall, then sorted and formatted for clearer analysis.

2.4 Data Analysis

```python
# Calculate averages
print("Average max temp:", monthly_df['max_temp'].mean())
print("Average min temp:", monthly_df['min_temp'].mean())

# Calculate median
print("Median max temp:", monthly_df['max_temp'].median())
print("Median min temp:", monthly_df['min_temp'].median())

# Highest and lowest temperatures
print("Highest temp:", monthly_df['max_temp'].max())
print("Lowest temp:", monthly_df['min_temp'].min())

# Total rainfall
print("Total rainfall:", monthly_df['rain'].sum())
```

Description: This code calculates key statistics such as average, median, maximum, minimum temperatures, and total rainfall to analyze overall weather trends.

2.5 Visualizations

```python
import matplotlib.pyplot as plt

# Line chart for temperature
plt.figure()
plt.plot(monthly_df['month_name'], monthly_df['max_temp'], marker='o')
plt.plot(monthly_df['month_name'], monthly_df['min_temp'], marker='o')

plt.title("Monthly Temperature in Qatar (2025)")
plt.xlabel("Month")
plt.ylabel("Temperature (°C)")
plt.xticks(rotation=45)

plt.show()


# Bar chart for rainfall
plt.figure()
plt.bar(monthly_df['month_name'], monthly_df['rain'])

plt.title("Monthly Rainfall in Qatar (2025)")
plt.xlabel("Month")
plt.ylabel("Rainfall (mm)")
plt.xticks(rotation=45)

plt.show()
```

Description: The charts show that temperatures in Qatar are highest during summer and lower in winter, while rainfall remains very low throughout the year.

3. Visualizations

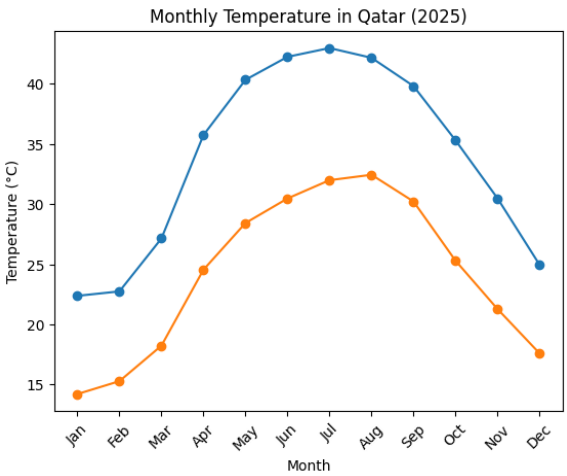

Figure 1

The line chart illustrates the monthly maximum and minimum temperatures in Qatar for 2025. It shows a clear increase in temperatures during the summer months, with peak values occurring between June and August, followed by a gradual decrease towards the end of the year. This reflects the typical hot climate of the region.

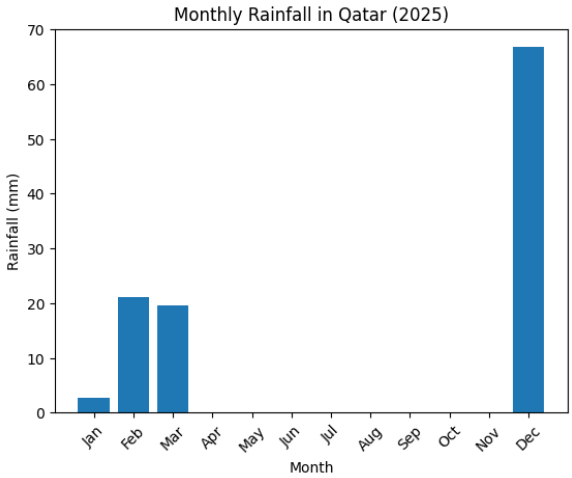

Figure 2

The bar chart represents the monthly rainfall levels in Qatar. The results indicate that rainfall is very limited and occurs only in a few months, which is consistent with the country's arid desert climate.

Reflection:

This task involved analyzing a structured weather dataset for Qatar in 2025, which included variables such as date, maximum and minimum temperature, and rainfall. The dataset was organized in a tabular format, making it suitable for structured data analysis.

SQL databases are designed to manage structured data using predefined schemas, tables, and relationships. This makes them highly suitable for this dataset, as the data follows a consistent format with clearly defined columns. For example, a government weather monitoring system could store daily temperature records in a SQL database and easily perform queries to calculate monthly averages or identify the hottest periods of the year. SQL ensures data integrity and supports efficient analytical queries, which are essential for accurate reporting and decision-making (Elmasri and Navathe, 2016).

In contrast, NoSQL databases are designed for flexibility and are better suited for unstructured or semi-structured data. For instance, a social media platform that stores user-generated content such as text, images, and videos would benefit from NoSQL due to its ability to handle varying data formats (Han, Kamber and Pei, 2011). However, this flexibility is not required for structured datasets like weather data.

In conclusion, SQL is the more appropriate storage model for this dataset due to its structured nature and the need for precise analysis, while NoSQL is more suitable for dynamic and unstructured data scenarios (Stonebraker, 2010).

References

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Pearson.

Han, J., Kamber, M. and Pei, J. (2011) *Data Mining: Concepts and Techniques*. 3rd edn. Burlington: Morgan Kaufmann.

Stonebraker, M. (2010) ‘SQL databases v. NoSQL databases’, *Communications of the ACM*, 53(4), pp. 10–11.